

An Enhanced Sub image Matching Algorithm for Binary Images

Azzam Sleit, Heba Saadeh, Ibraheem Al-Dhamari, Afaf Tareef
Department of Computer Science, King Abdulla II School for Information Technology
University of Jordan, Amman, Jordan

azzam.sleit@ju.edu.jo, heba.saadeh@ju.edu.jo, ibr_ex@yahoo.com, a.tareef@yahoo.com

Abstract: - Several applications in image processing, computer vision and geographic information systems tend to deal with binary images. Few matching algorithms which search for a small image inside a large one exist in the literature. This article presents an enhancement for the Chain Code based exact match algorithm which transforms the matrices into reduced sizes [2]. The proposed enhancement utilizes a user input threshold value; i.e., miss-match or error value to allow for possible noise handling. Experiments show that the proposed modification discovers more patterns inside the search space.

Key-Words: - Binary image matching, Chain code based algorithm, Approximation, Best match, Exact match.

1. Introduction

A binary image is a digital image that consists of two colors, usually black and white. One of the colors represents the objects as a foreground color while the other represents the background of the scenery. Binary images are also called bi-level, two level, monochrome, or black and white images [1, 7].

A wide range of applications concerned with object tracking and recognition tend to process the original images into binary matrices for simplicity [3, 4, 5]. A naïve Brute-Force algorithm can be used to detect the existence of an image inside a larger one with a worst case time complexity $O(m^2 n^2)$ and $O(n^2)$ in the best case, where n is the number of rows (or columns) of the larger image, and m is the number of rows (or columns) of the sub-image [2].

Sleit et. al. presented a Chain Code [6] based exact match algorithm (CCEM algorithm) for detecting the existence of a matrix pattern inside a larger one. The CCEM algorithm assumes a preprocessing transformation phase which takes a main matrix (MM) and a sub-matrix (SM) and transforms them into vectors TMM and TSM respectively. TMM has the same number of row as MM and similarly TSM and SM. For each row in the original matrix, a string is created in the transformed vector consisting of the number of consecutive zeros followed by the number of consecutive ones and so

on for that particular row. Considering the following matrix A: 4X4, the first row starts with three zeros followed by a single one. Therefore, the first component of vector TA will be '3, 1'. Likewise, the second component will be '2, 2'. The third row of A does not start with zero but has all ones. Therefore, it will be transformed into '0, 4'. Finally, the fourth row starts with a single one, followed by two zeros, followed by a single one. It will be transformed into '0, 1, 2, 1'.

$$A = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix} \quad TA = \begin{pmatrix} 3 & 1 \\ 2 & 2 \\ 0 & 4 \\ 0 & 1 & 2 & 1 \end{pmatrix}$$

The transformation phase obviously reduces the size of the images database to get them prepared for future match queries. For images containing actual objects, the transformed images can reach half the size of the original images [2].

Section 2 of this article discusses the CCEM algorithm. Section 3 proposes an enhancement for CCEM to find approximate matches. Section 4 demonstrates examples of experiments which show the merits of the enhancement in handling the noise issue. The article concludes by section 5 with summary and future work.

2. The Chain Code Based Exact Match (CCEM) Algorithm

The CCEM algorithm searches for an exact match of image SM inside a larger image MM, using their corresponding transformed vectors TSM and TMM respectively.

Fig. 1 presents the flow chart of finding the first matched element between SM and MM [2]. When detecting the first matched element, the function “Return offset in original Main Matrix” as described in Fig. 2 is made to determine the exact row and column numbers of SM in MM.

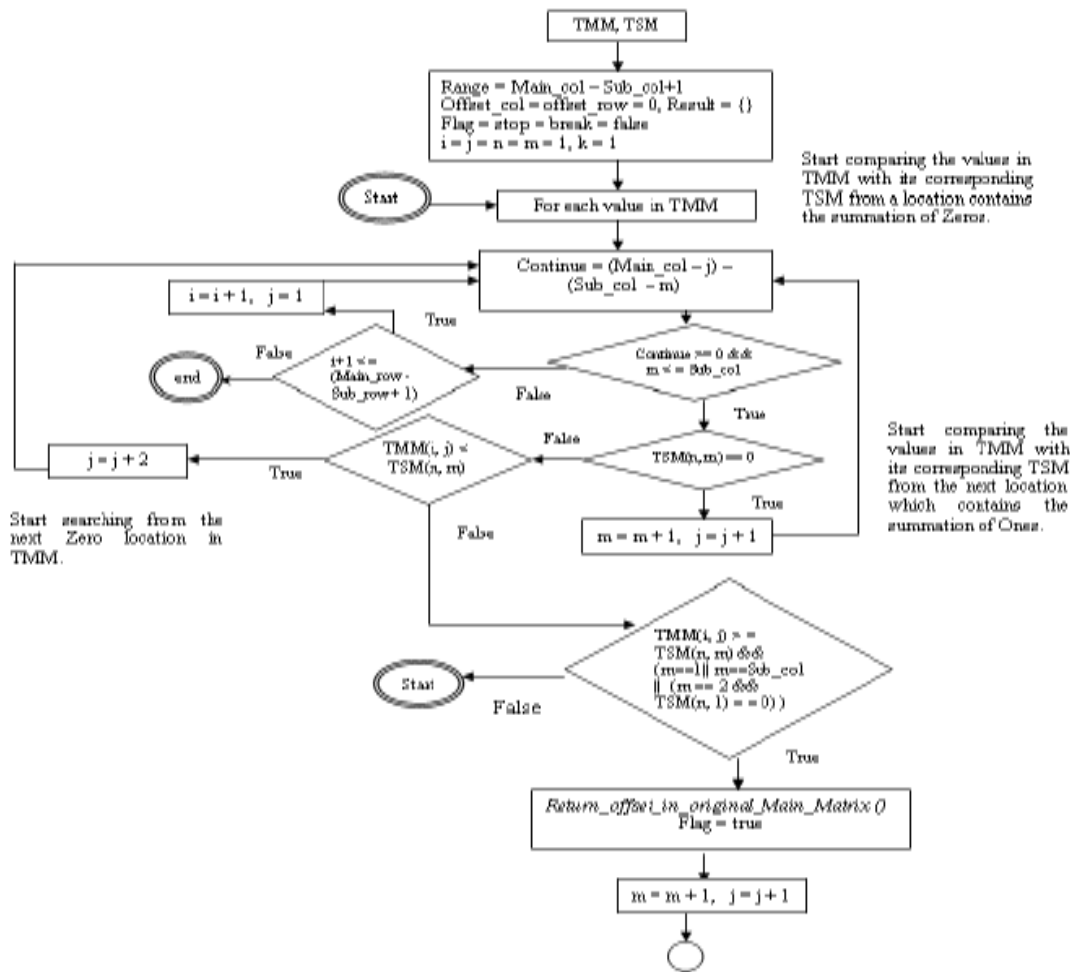


Fig 1. CCEM - Flow chart for finding the first point of match in TMM.

The search process starts by detecting the first pixel match and continues comparing in the current row pixel by pixel. When it reaches the end of the current row, it determines the offset of the column in the next row [2]. Search is terminated when one of the following two cases occurs:

- If the last element of TSM is reached, then a sub-matrix match is found.
- If the value of TMM is less than the corresponding one in TSM, then a match is not found.

Algorithm: **Return_offset_in_original_Main_Matrix ()**
 // find the exact row and column in the original Main Matrix.

```

1. counter = 0;
2. if ( Flag == false)
3.     next = j; // to calculate the next position from
                 which we will continue searching.
4. if (j > 1)
5.     { for (d = 1 to j - 1)
          // find the summation of previous values.
6.         counter = counter + TMM (i , d); }
7. else
8.     counter = 0;
9. offset_row = i;
10. offset_col = TMM ( i , j) - TSM (n , m) + counter + 1;

// we add 1 because the matrix's index starts from 1.
11. Return offset_row, offset_col;

```

Fig 2. Return offset in original Main Matrix Function

3. The Chain Code Based Approximate Match (CCAM) Algorithm

The existence of noisy pixels in an image during its acquisition motivates the search for approximate matching techniques. In this article, we introduce the CCAM algorithm which incorporates a percentage of mismatch or error value in the search process which gives CCAM the flexibility over CCEM for handling mismatch pixels. Similar to CCEM, CCAM performs the transformation preprocessing phase to gain the advantage of size reduction by 50% on average and thereby reduce the number of comparisons. The search phase starts by comparing the pixels in SM and MM, pixel by pixel. In case of a mismatch, the error value count is incremented by one, and the comparison process continues while the error count has not reached a threshold value. Fig. 3 describes the search process in CCAM.

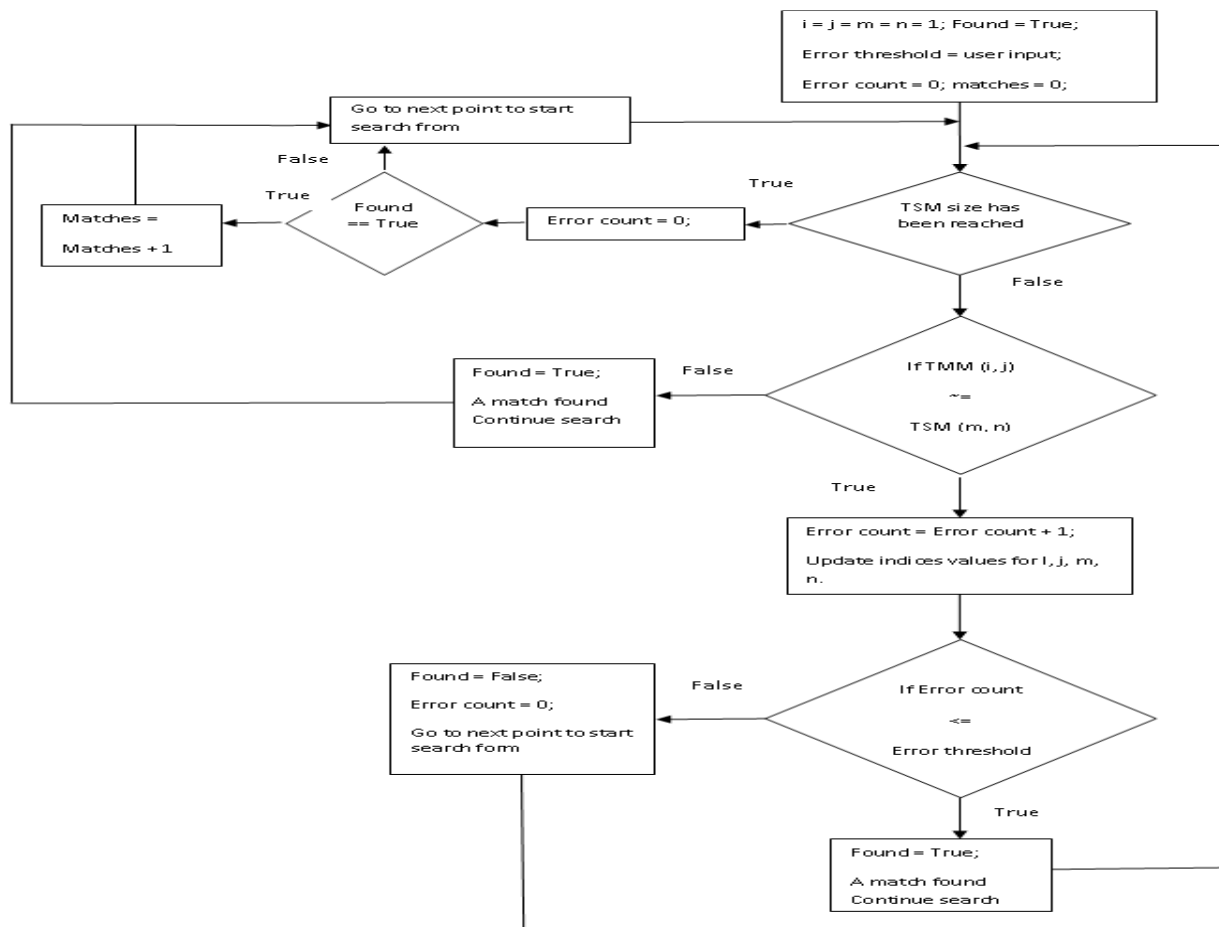
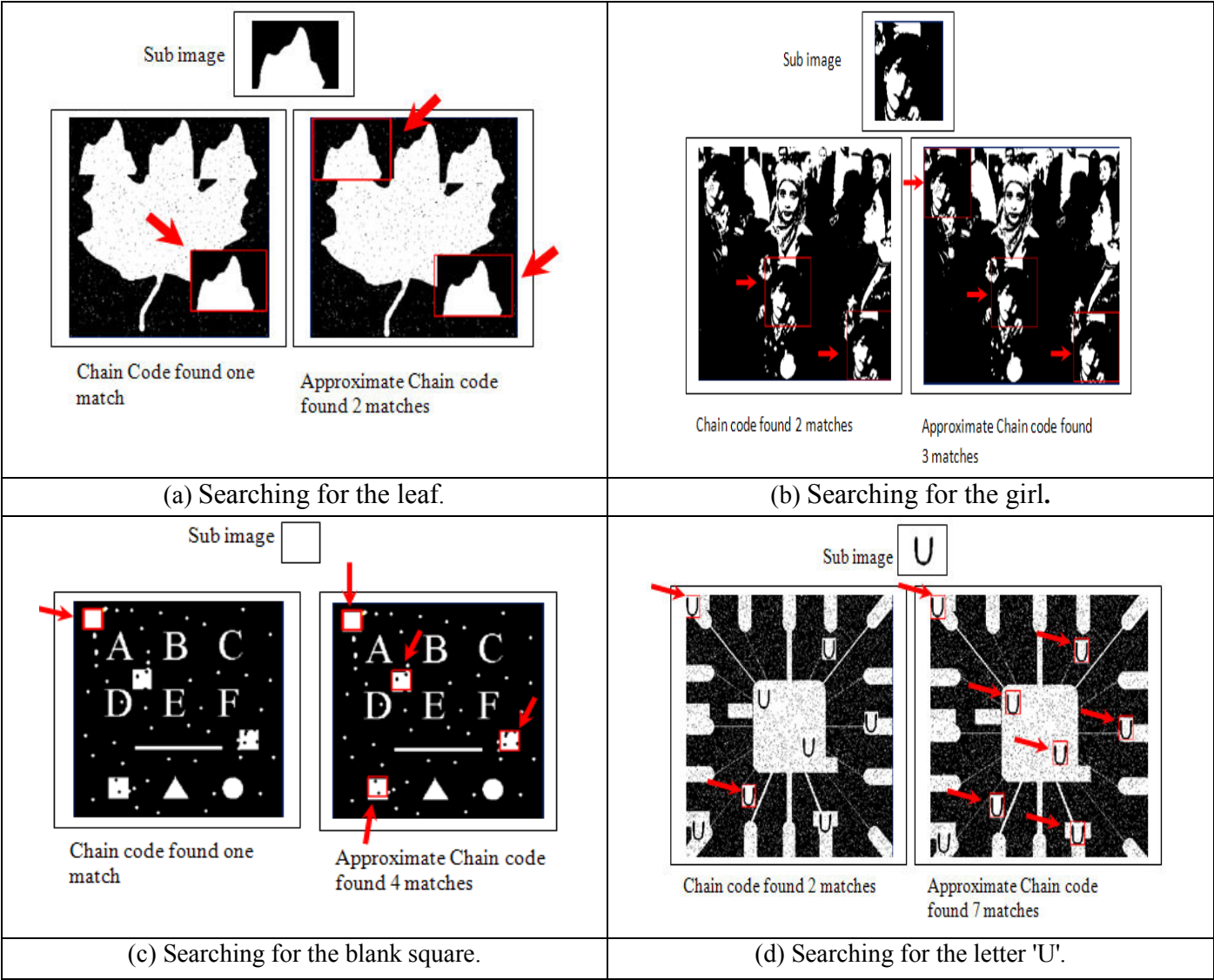


Fig 3. The CCAM search process to find similar images.

Choosing a small error threshold value causes the search algorithm to be less tolerant for noise. However, selecting a large error threshold value may cause irrelevant images to be retrieved. The approximation approach adds flexibility to the CCEM algorithm by giving the user the ability to provide a suitable error threshold value which tends to be more suitable for real life applications.

4. Experiments and Results

The CCAM and CCEM algorithms were implemented using MATLAB. For comparison purposes, both algorithms ran on various types of images with different sizes. As expected the CCAM algorithm found more matches than the CCEM due to the fact that CCAM is more tolerant to noise, as illustrated in Fig 4.



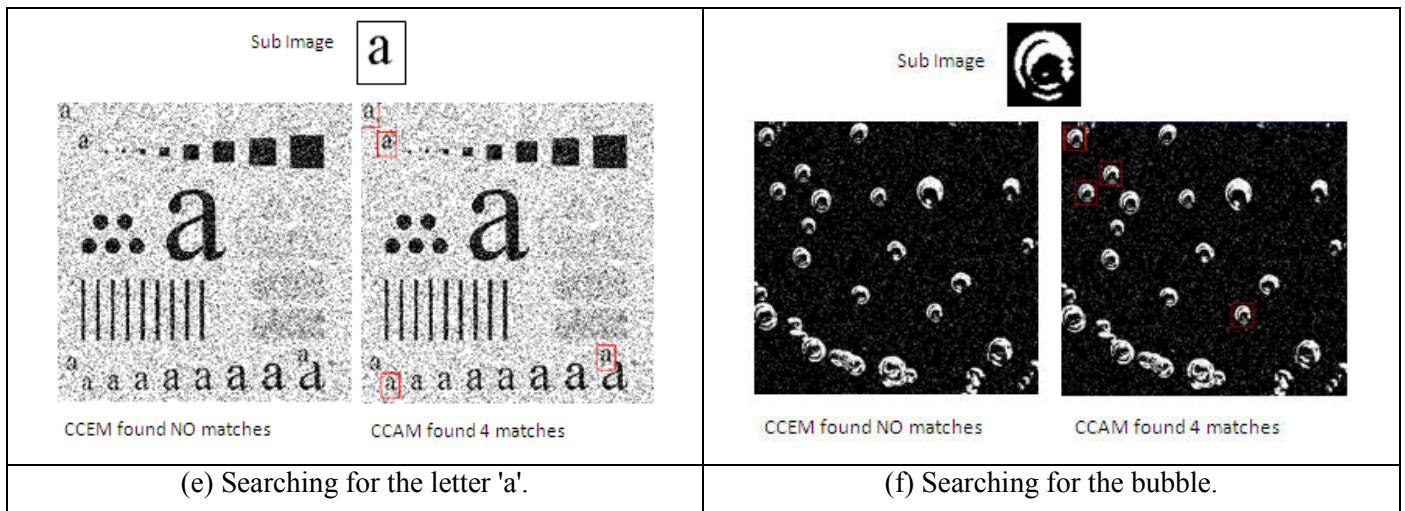


Fig 4. Experimental comparison between the CCEM and CCAM algorithms.

5. Conclusion

This article proposes an enhancement for the chain code based exact match algorithm for detecting the existence of a matrix pattern inside a larger one. The enhancement allows for finding approximate matches for the test pattern that do not exceed a specific error value. The enhanced algorithm is more suitable for real life applications that tend to deal with images or patterns which may include noise. Experiments confirm that the inclusion of an error difference in the matching process gives more desirable matching results.

The proposed enhancement does not handle the issue of rotating and scaling the sub matrix before matching it with the main matrix. This is an avenue for future research which will further enhance the Chain Code based matching technique.

References:

- [1] Rafael C. Gonzalez and Richard E. Woods, 2008, Digital Image Processing, prentice Hall, 3rd edition.
- [2] Sleit, A., AlMobaideen, W., Qatawneh, M., Saadeh, H., 2008, Efficient Processing for Binary Submatrix Matching, American Journal of Applied Sciences, Vol. 6, No.1, pp. 78-88.
- [3] Knuth, D.E, Morris, J.H., Pratt, V.R., 1977, Fast Pattern Matching in Strings, SIAM Journal on Computing, Vol. 6, No. 2, pp. 323-350.
- [4] Boyer, R. S. and Moore, J.S., 1977, A Fast String Searching Algorithm, Communications of the ACM, Vol. 20, No. 10, pp. 762 - 772.
- [5] Smith, P., 1991, Experiments with a Very Fast Substring Search Algorithm, Software – Practice and Experience, Vol. 21, No. 10, pp. 1065–1074.
- [6] Mei-Chen, Y., Huang, Y. and Wang, J., 2002, Scalable Ideal-Segmented Chain Coding, IEEE International Conference on Image Processing, pp. 197-200.
- [7] Bronson, R., Schaum, S., 1989, Outline of Theory and Problems of Matrix Operations, McGraw Hill.